

Introduction to Computer Programming with Python

Amir Jamalian



Gisma
University
of Applied
Sciences



Computers

- The uses of computers are almost limitless in our everyday lives

- Writing papers
- Searching for articles
- Sending emails
- Participating in online classes and meetings
- Analyzing data
- Shopping online
- Playing computer games



- Why can't we such things with other devices/machines?

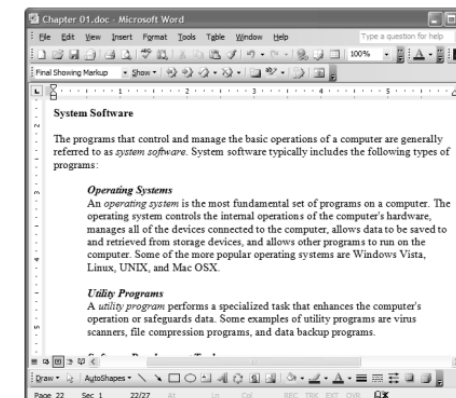
- Such as a shaver?





Programs

- Computers can be programmed
 - Therefore, they can do such a wide variety of things
 - This means that computers are not designed to do just one job
 - But to do any job that their programs tell them to do
- A program is a set of instructions that a computer follows to perform a task
 - For example, the figure shows screens from two commonly used programs
 - Microsoft Word is a word processing program
 - That allows you to create, edit, and print documents with your computer
 - Adobe Photoshop is an image editing program
 - That allows you to work with graphic images
 - Such as photos taken with your digital camera





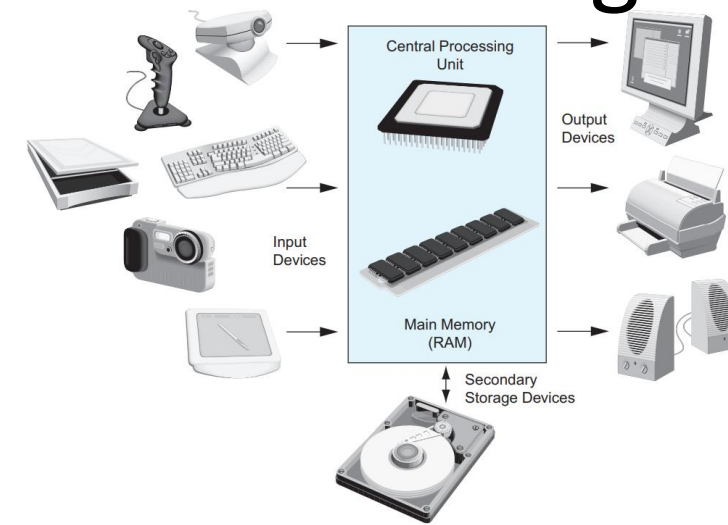
Programmers

- Programs are commonly referred to as software
 - Software is essential to a computer because it controls everything the computer does
 - All of the software that we use to make our computers useful is created by individuals working
 - As programmers or software developers
- A programmer (or software developer) can design, create, and test computer programs
 - Computer programming is an exciting and rewarding career
 - Today, you will find programmers' work used in business, medicine, government, law enforcement, agriculture, academics, entertainment, and many other fields



Hardware

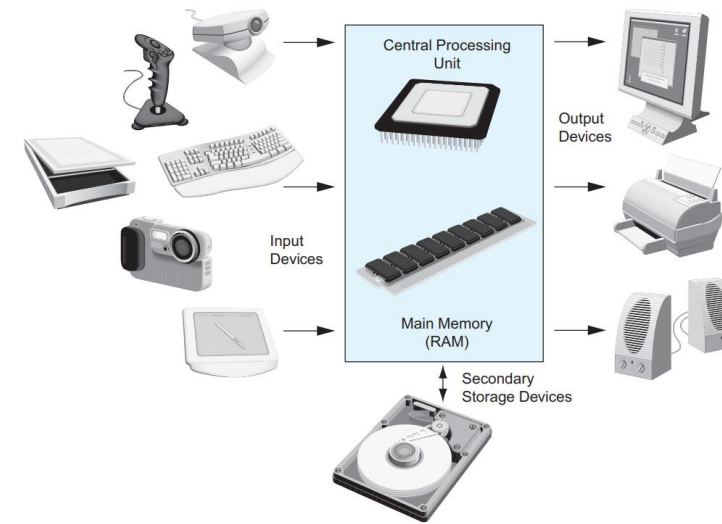
- Hardware refers to all of the physical devices, or components, that a computer is made of
 - A computer is not one single device, but a system of devices that all work together
 - Each device in a computer plays its own part
 - Like the different instruments in a symphony orchestra
- A typical computer system consists of the following major components
 - The central processing unit (CPU)
 - Main memory
 - Secondary storage devices
 - Input devices
 - Output devices





The CPU

- The computer is running or executing the program
 - When a computer is performing the tasks that a program tells it to do
- The central processing unit, or CPU, is the part of a computer that actually runs programs
 - The CPU is the most important component in a computer
 - Because without it, the computer could not run software

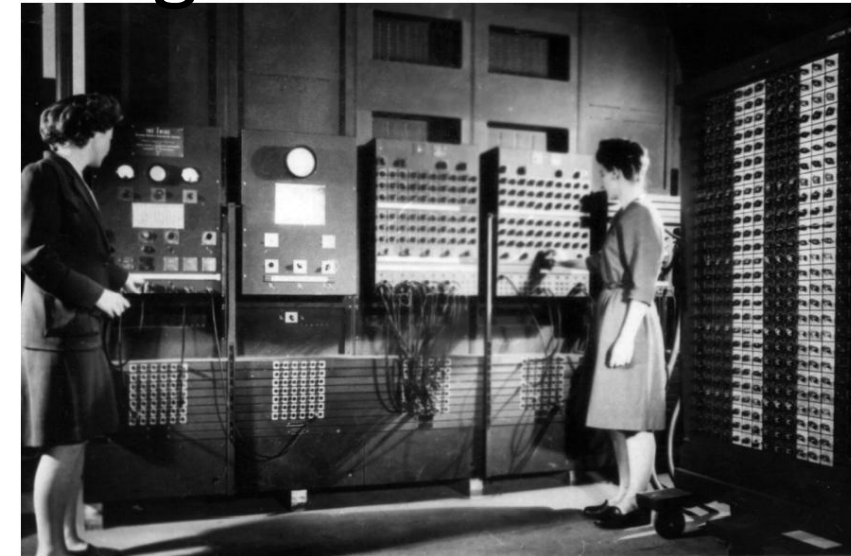




The CPU

- In the earliest computers, CPUs were huge devices

- Made of electrical and mechanical components such as vacuum tubes and switches
- The ENIAC was built in 1945
 - To calculate artillery ballistic tables for the U.S. Army
 - It is considered as the world's first programmable electronic computer
 - This machine was
 - 8 feet tall
 - 100 feet long
 - Weighed 30 tons
 - It was primarily one big CPU
 - The two women in the photo are working with the historic ENIAC computer



- Today, CPUs are small chips known as microprocessors

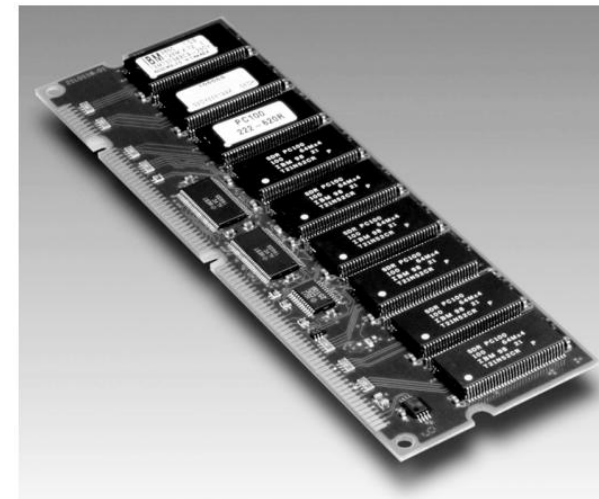
- Microprocessors are
 - Much smaller than the old electromechanical CPUs in early computers
 - Much more powerful
- The technician in the photo holds a modern microprocessor





Main Memory

- You can think of main memory as the computer's work area
 - This is where the computer stores
 - A program while the program is running
 - The data that the program is working with
- Main memory is commonly known as random-access memory, or RAM
 - It is called this because the CPU is able to quickly access data
 - Stored at any random location in RAM
 - RAM is usually a volatile type of memory
 - That is used only for temporary storage while a program is running
 - When the computer is turned off, the contents of RAM are erased
 - Inside your computer, RAM is stored in chips





Secondary Storage Devices

- Secondary storage is a type of memory that can hold data for long periods of time

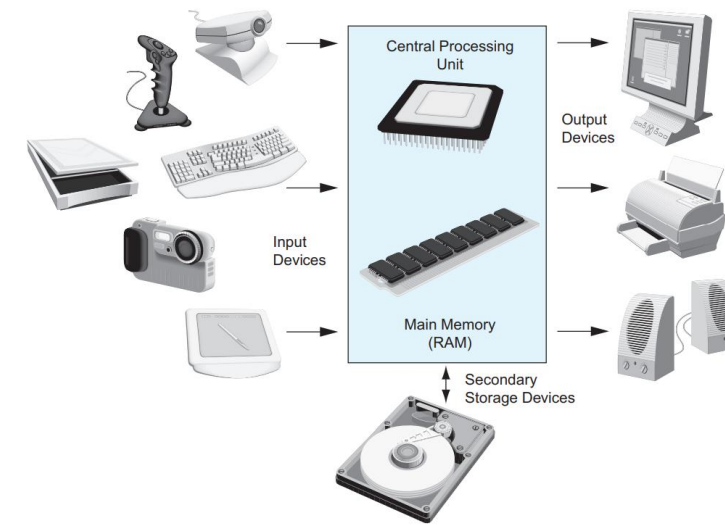
- Even when there is no power to the computer
- Programs are normally stored in secondary memory and loaded into main memory as needed
- The most common type of secondary storage device is the disk drive
 - A disk drive stores data by magnetically encoding it onto a circular disk
 - External disk drives are also available
 - Which connect to one of the computer's communication ports
- USB drives are small devices that plug into the computer's USB (universal serial bus) port
 - They appear to the system as a disk drive
 - These drives do not actually contain a disk, however
 - They store data in a special type of memory known as flash memory
- Optical devices such as the CD (compact disc) and the DVD (digital versatile disc) are also popular for data storage
 - Data is not recorded magnetically on an optical disc, but is encoded as a series of pits on the disc surface
 - CD and DVD drives use a laser to detect the pits and thus read the encoded data





Input and Output Devices

- An input device is the component that collects the data and sends it to the computer
 - Common input devices are the keyboard, mouse, scanner, microphone, and digital camera
 - Disk drives and optical drives can also be considered input devices
 - Because programs and data are retrieved from them and loaded into the computer's memory
- The output device formats and presents the data that is sent by the computer
 - Common output devices are video displays and printers
 - Disk drives and CD recorders can also be considered output devices
 - Because the system sends data to them in order to be saved





Software

- Everything that a computer does is under the control of software
 - From the time you turn the power switch on until you shut the system down
 - If a computer is to function, software is not optional
- There are two general categories of software
 - System software
 - Application software



System Software

- System software is the programs that control and manage the basic operations of a computer

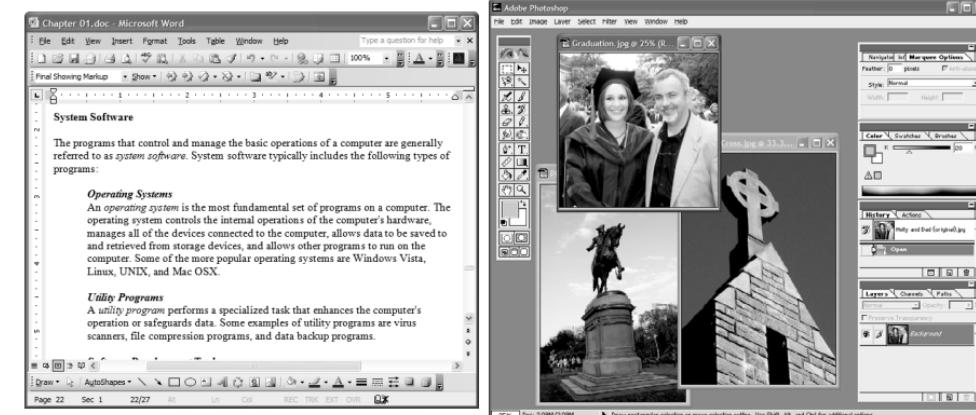
- Operating Systems
 - An operating system is the most fundamental set of programs on a computer
 - The operating system
 - Controls the internal operations of the computer's hardware
 - Manages all of the devices connected to the computer
 - Allows data to be saved to and retrieved from storage devices
 - Allows other programs to run on the computer
- Utility Programs
 - A utility program performs a specialized task that enhances the computer's operation or safeguards data
 - Examples of utility programs are virus scanners, file compression programs, and data backup programs
- Software Development Tools
 - Software development tools are the programs that programmers use to create, modify, and test software
 - Assemblers, compilers, and interpreters are examples of programs that fall into this category





Application Software

- Application software is programs that make a computer useful for everyday tasks
 - These are the programs that people normally spend most of their time running on their computers
 - Some examples of application software are spreadsheet programs, email programs, web browsers, and game programs

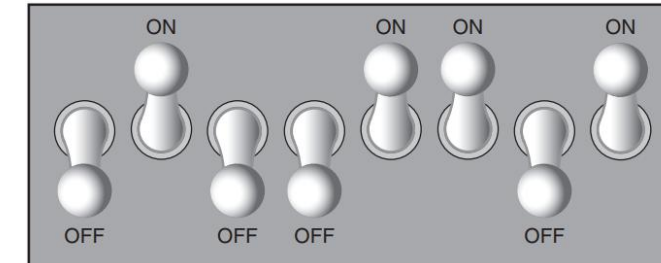




How Computers Store Data

- A computer's memory is divided into tiny storage locations known as bytes

- One byte is only enough memory to store a letter of the alphabet or a small number
- In order to do anything meaningful, a computer has to have lots of bytes
- Most computers today have millions, or even billions, of bytes of memory



- Each byte is divided into eight smaller storage locations known as bits

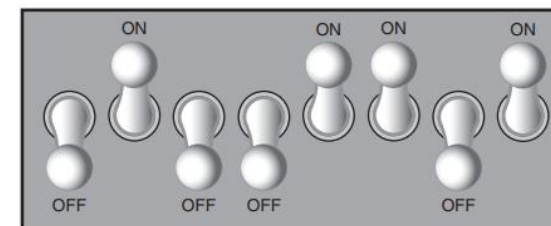
- The term bit stands for binary digit
- Computer scientists usually think of bits as tiny switches that can be either on or off
 - Bits aren't actual "switches," however, at least not in the conventional sense
- In most computer systems, bits are tiny electrical components that can hold either a positive or a negative charge



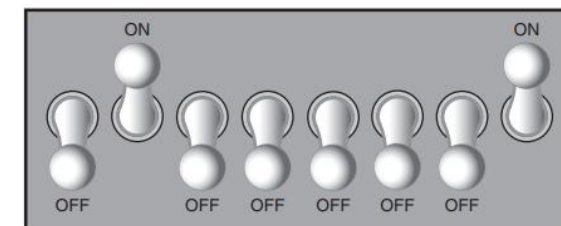
How Computers Store Data

- The computer sets the eight bits to an on/off pattern that represents the data

- When a piece of data is stored in a byte
- For example
 - The pattern shown on the left shows how the number 77 would be stored in a byte
 - The pattern on the right shows how the letter A would be stored in a byte



The number 77 stored in a byte.



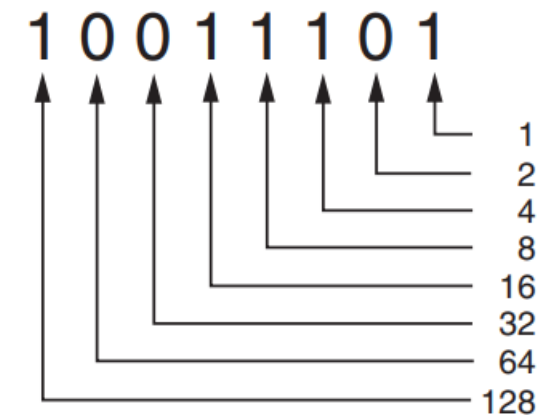
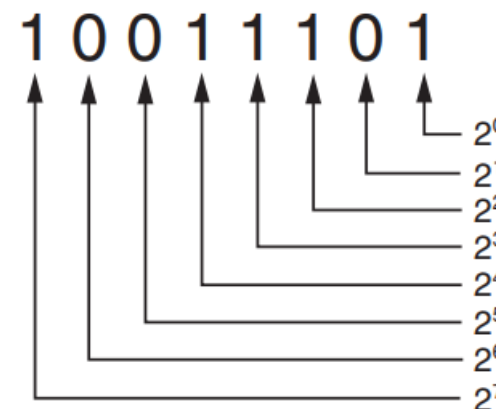
The letter A stored in a byte.



Storing Numbers

- A bit can be used in a very limited way to represent numbers
 - Depending on whether the bit is turned on or off, it can represent one of two different values
 - In computer systems, a bit that is turned off represents the number 0 and a bit that is turned on represents the number 1
 - This corresponds perfectly to the binary numbering system
 - In the binary numbering system (or binary, as it is usually called) all numeric values are written as sequences of 0s and 1s
- The position of each digit in a binary number has a value assigned to it

- Starting with the rightmost digit and moving left
 - The position values are 2^0 , 2^1 , 2^2 , 2^3 , and so forth
 - The position values are 1, 2, 4, 8, and so forth

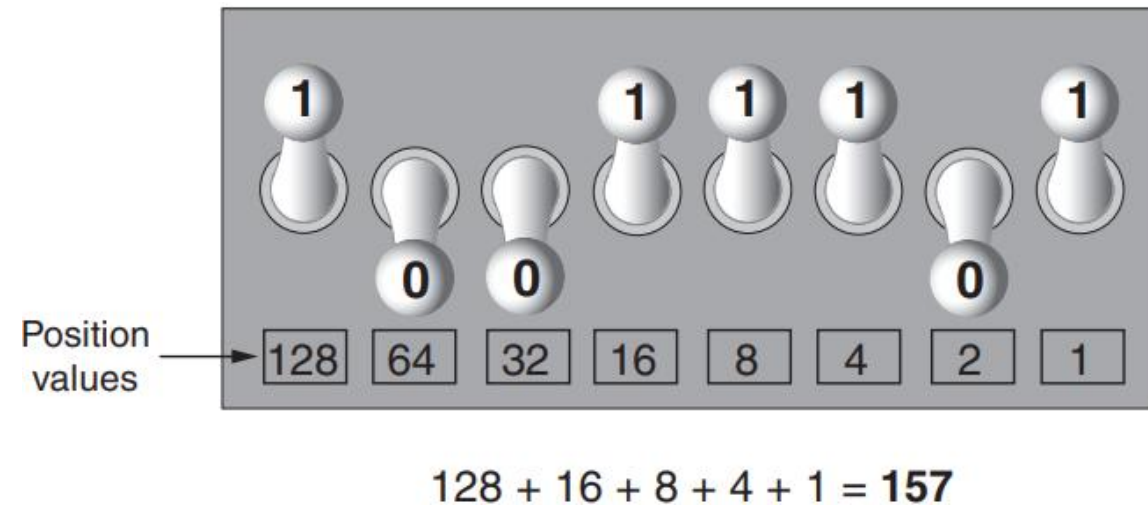
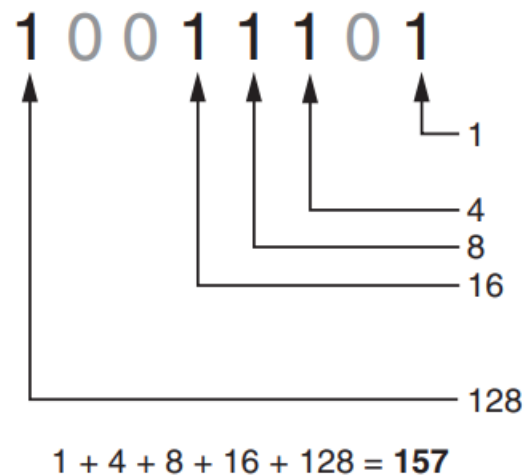




Storing Numbers

- To determine the value of a binary number you simply add up the position values of all the 1s

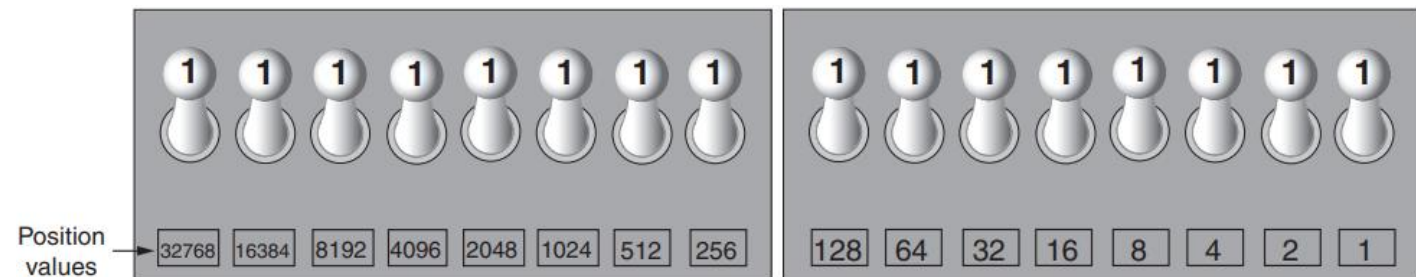
- In the binary number 10011101, the position values of the 1s are 1, 4, 8, 16, and 128
- The sum of all of these position values is 157
- So, the value of the binary number 10011101 is 157





Storing Numbers

- The largest value that can be stored in a byte is 255
 - Because $1+2+4+8+16+32+64+128=255$
 - This limit exists because there are only eight bits in a byte
- We should use more than one byte to store larger numbers
 - For example, the maximum value that can be stored in two bytes is 65,535



$$32768 + 16384 + 8192 + 4096 + 2048 + 1024 + 512 + 256 + 128 + 64 + 32 + 16 + 8 + 4 + 2 + 1 = 65535$$



Storing Characters

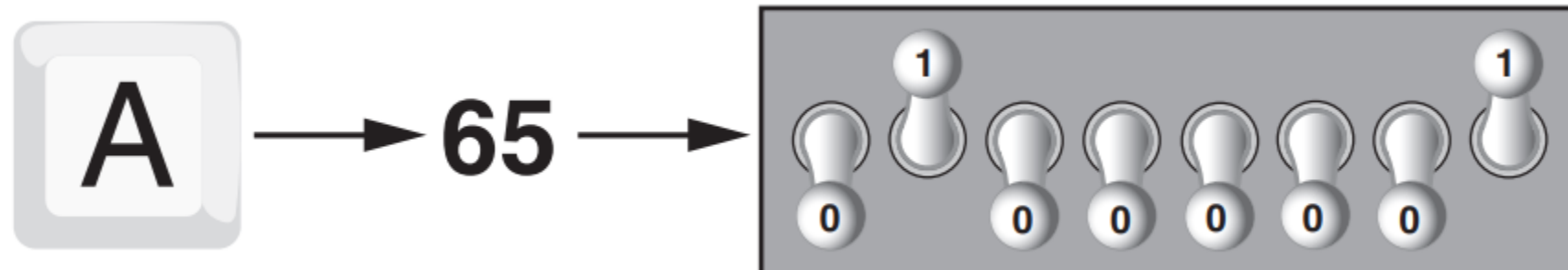
- Any piece of data that is stored in a computer's memory must be stored as a binary number
 - That includes characters, such as letters and punctuation marks
 - When a character is stored in memory, it is first converted to a numeric code
 - The numeric code is then stored in memory as a binary number
- Over the years, different coding schemes have been developed
 - To represent characters in computer memory



Storing Characters

- Historically, the most important of these coding schemes is ASCII

- Which stands for the American Standard Code for Information Interchange
- ASCII is a set of 128 numeric codes
 - That represent the English letters, various punctuation marks, and other characters
- The ASCII character set was developed in the early 1960s
 - It was eventually adopted by most all computer manufacturers
- For example, the ASCII code for the uppercase letter A is 65
 - When you type an uppercase A on your computer keyboard, the number 65 is stored in memory
 - As a binary number, of course





Storing Characters

- ASCII is limited however because it defines codes for only 128 characters
 - To remedy this, the Unicode character set was developed in the early 1990s
- Unicode is an extensive encoding scheme that is compatible with ASCII
 - But can also represent characters for many of the languages in the world
 - Today, Unicode is quickly becoming the standard character set used in the computer industry



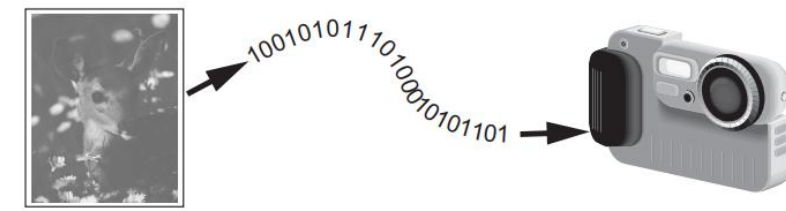
Advanced Number Storage

- Computers are able to store negative numbers and real numbers in memory
 - But to do so they use encoding schemes along with the binary numbering system
 - Negative numbers are encoded using a technique known as two's complement
 - Real numbers are encoded in floating-point notation
- You don't need to know how these encoding schemes work
 - Only that they are used to convert negative numbers and real numbers to binary format



Other Types of Data

- Computers are often referred to as digital devices
 - The term digital can be used to describe anything that uses binary numbers
 - Digital data is data that is stored in binary
 - A digital device is any device that works with binary data
- Computers also work with many other types of digital data
 - For example, consider the pictures that you take with your digital camera
 - These images are composed of tiny dots of color known as pixels
 - The term pixel stands for picture element
 - Each pixel in an image is converted to a numeric code that represents the pixel's color
 - The numeric code is stored in memory as a binary number

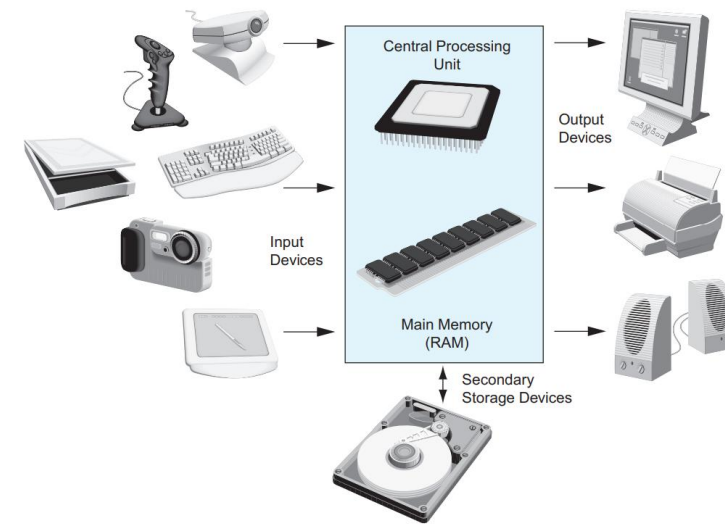




How a Program Works

• The CPU is the most important component in a computer

- Because it is the part of the computer that runs programs
- Sometimes the CPU is called the “computer’s brain,” and is described as being “smart”
 - Although these are common metaphors, you should understand that the CPU is not a brain, and it is not smart
- The CPU is an electronic device that is designed to do specific things
- In particular, the CPU is designed to perform simple operations on pieces of data
 - Reading a piece of data from main memory
 - Adding two numbers
 - Subtracting one number from another number
 - Multiplying two numbers
 - Dividing one number by another number
 - Moving a piece of data from one memory location to another
 - Determining whether one value is equal to another value





How a Program Works

- A program is nothing more than a list of instructions
 - That cause the CPU to perform operations
 - Each instruction in a program is a command that tells the CPU to perform a specific operation
 - The CPU does nothing on its own
 - It has to be told what to do
 - That is the purpose of a program
- 10110000
 - To you and me, this is only a series of 0s and 1s
 - To a CPU, however, this is an instruction to perform an operation
 - It is written in 0s and 1s because CPUs only understand instructions that are written in machine language
 - Machine language instructions always have an underlying binary structure



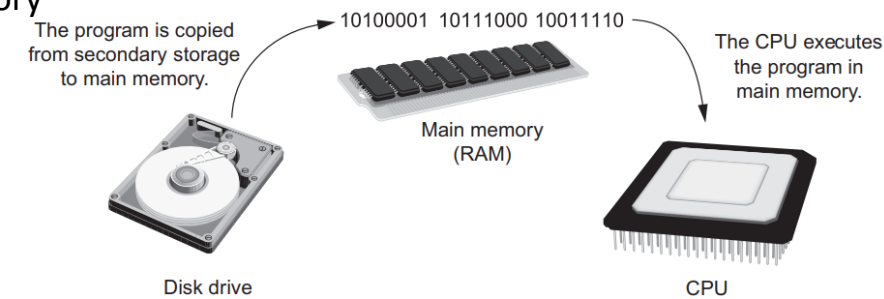
How a Program Works

- A machine language instruction exists
 - For each operation that a CPU is capable of performing
 - For example, there is an instruction for
 - Adding numbers
 - Subtracting one number from another
 - The entire set of instructions that a CPU can execute is known as the CPU's instruction set
- A program usually contains thousands or even millions of machine language instructions
 - Because the operations that a CPU knows how to perform are so basic in nature
 - A meaningful task can be accomplished only if the CPU performs many operations
 - For example, the CPU will have to perform a large number of instructions, carried out in the proper sequence
 - If you want your computer to calculate the amount of interest that you will earn from your savings account this year



How a Program Works

- Programs are usually stored on a secondary storage device
 - Such as a disk drive
- The program has to be copied into main memory, or RAM, each time the CPU executes it
 - For example, suppose you have a word processing program on your computer's disk
 - To execute the program you use the mouse to double-click the program's icon
 - This causes the program to be copied from the disk into main memory
 - Then, the computer's CPU executes the copy of the program that is in main memory

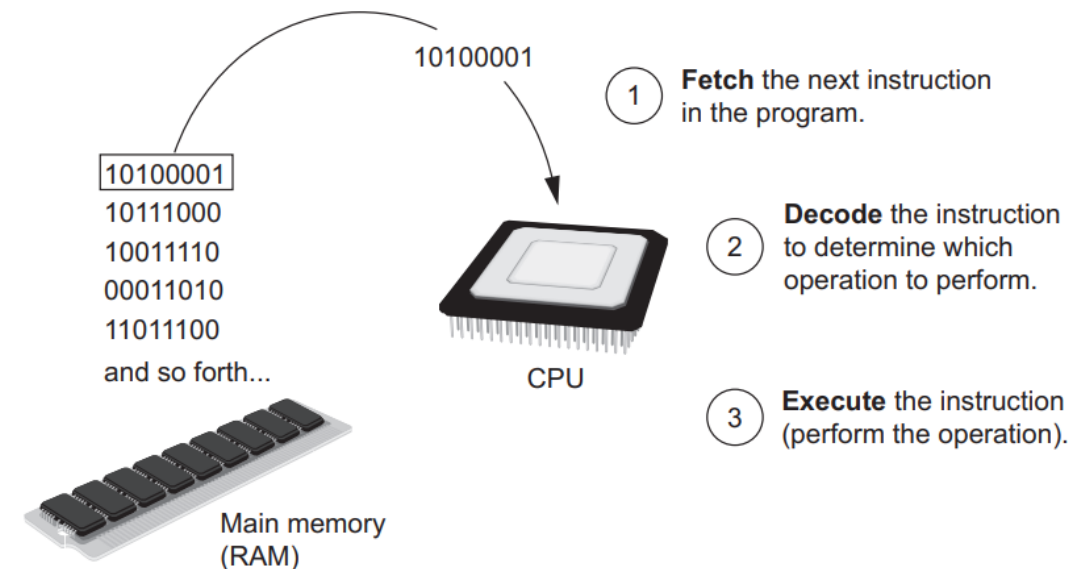




How a Program Works

- The CPU is engaged in a process that is known as the fetch-decode-execute cycle
 - When it executes the instructions in a program
- This cycle is repeated for each instruction in the program

- Fetch
- Decode
- Execute





From Machine Language to Assembly Language

- Computers can only execute programs that are written in machine language
 - But it is impractical for people to write programs in machine language
- Programming in machine language would also be very difficult
 - A program can have thousands or even millions of binary instructions
 - Writing such a program would be very tedious and time consuming
 - Putting a 0 or a 1 in the wrong place will cause an error



From Machine Language to Assembly Language

- For this reason, assembly language was created
 - In the early days of computing as an alternative to machine language
 - The first assembly language was most likely that developed in the 1940s at Cambridge University
 - For use with a historic computer known as the EDSAC
 - Instead of using binary numbers for instructions, assembly language uses short words
 - That are known as mnemonics
- Assembly language programs cannot be executed by the CPU, however
 - The CPU only understands machine language
 - So, a special program known as an assembler is used
 - To translate an assembly language program to a machine language program
 - The machine language program can then be executed by the CPU

Assembly language
program

```
mov eax, Z
add eax, 2
mov Y, eax

and so forth...
```

Assembler

Machine language
program

```
10100001
10111000
10011110

and so forth...
```



High-Level Languages

- Assembly language is not without difficulties
 - Although assembly language makes it unnecessary to write binary machine language instructions
 - Assembly language is primarily a direct substitute for machine language
 - Like machine language, it requires that you know a lot about the CPU
 - Assembly language also requires that you write a large number of instructions for even the simplest program
- Assembly language is referred to as a low-level language
 - Because assembly language is so close in nature to machine language

Assembly language
program

```
mov eax, Z
add eax, 2
mov Y, eax
and so forth...
```

Assembler

Machine language
program

```
10100001
10111000
10011110
and so forth...
```



High-Level Languages

- In the 1950s, high-level languages began to appear

- As a new generation of programming languages
- A high-level language allows you to create powerful and complex programs
 - Without knowing how the CPU works
 - Without writing large numbers of low-level instructions
- In addition, most high-level languages use words that are easy to understand

- DISPLAY "Hello world"

- For example, this is an instruction to display the message "Hello world" on the computer screen in COBOL
 - Which was one of the early high-level languages created in the 1950s
- Doing the same thing in assembly language would require several instructions
 - And an intimate knowledge of how the CPU interacts with the computer's output device
- High-level languages allow programmers to concentrate on the tasks they want to perform with their programs
 - Rather than the details of how the CPU will execute those programs



High-Level Languages

- Thousands of high-level languages have been created

- Since the 1950s

Language	Description
Ada	Ada was created in the 1970s, primarily for applications used by the U.S. Department of Defense. The language is named in honor of Countess Ada Lovelace, an influential and historic figure in the field of computing.
BASIC	Beginners All-purpose Symbolic Instruction Code is a general-purpose language that was originally designed in the early 1960s to be simple enough for beginners to learn. Today, there are many different versions of BASIC.
FORTRAN	FORMula TRANslator was the first high-level programming language. It was designed in the 1950s for performing complex mathematical calculations.
COBOL	Common Business-Oriented Language was created in the 1950s, and was designed for business applications.
Pascal	Pascal was created in 1970, and was originally designed for teaching programming. The language was named in honor of the mathematician, physicist, and philosopher Blaise Pascal.
C and C++	C and C++ (pronounced “c plus plus”) are powerful, general-purpose languages developed at Bell Laboratories. The C language was created in 1972 and the C++ language was created in 1983.
C#	Pronounced “c sharp.” This language was created by Microsoft around the year 2000 for developing applications based on the Microsoft .NET platform.
Java	Java was created by Sun Microsystems in the early 1990s. It can be used to develop programs that run on a single computer or over the Internet from a web server.
JavaScript	JavaScript, created in the 1990s, can be used in web pages. Despite its name, JavaScript is not related to Java.
Python	Python, the language we use in this book, is a general-purpose language created in the early 1990s. It has become popular in business and academic applications.
Ruby	Ruby is a general-purpose language that was created in the 1990s. It is increasingly becoming a popular language for programs that run on web servers.
Visual Basic	Visual Basic (commonly known as VB) is a Microsoft programming language and software development environment that allows programmers to create Windows-based applications quickly. VB was originally created in the early 1990s.



Key Words, Operators, and Syntax: An Overview

- Each high-level language has its own set of predefined words that the programmer must use to write a program
 - The words that make up a high-level programming language are known as key words or reserved words
 - Each key word has a specific meaning and cannot be used for any other purpose
 - The table shows all of the Python key words

and	del	from	not	while
as	elif	global	or	with
assert	else	if	pass	yield
break	except	import	print	
class	exec	in	raise	
continue	finally	is	return	
def	for	lambda	try	



Key Words, Operators, and Syntax: An Overview

- Programming languages have operators that perform various operations on data
 - In addition to key words
 - For example, all programming languages have math operators that perform arithmetic
 - In Python, as well as most other languages, the + sign is an operator that adds two numbers
 - $12 + 75$
- Each language also has its own syntax
 - Which is a set of rules that must be strictly followed when writing a program
 - The syntax rules dictate how key words, operators, and various punctuation characters must be used in a program
 - When you are learning a programming language, you must learn the syntax rules for that particular language



Compilers and Interpreters

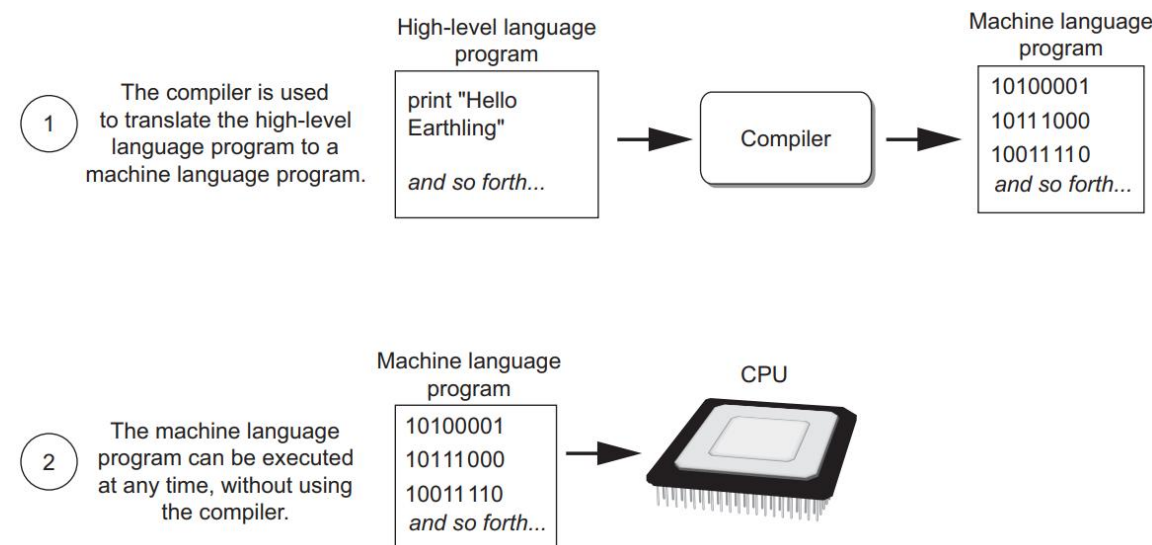
- Programs that are written in a high-level language must be translated into machine language
 - Because the CPU understands only machine language instructions
- The programmer will use either a compiler or an interpreter to make the translation
 - Depending on the language that a program has been written in



Compilers and Interpreters

- A compiler is a program that translates a high-level language program into a separate machine language program

- The machine language program can then be executed any time it is needed
- As shown in the figure, compiling and executing are two different processes

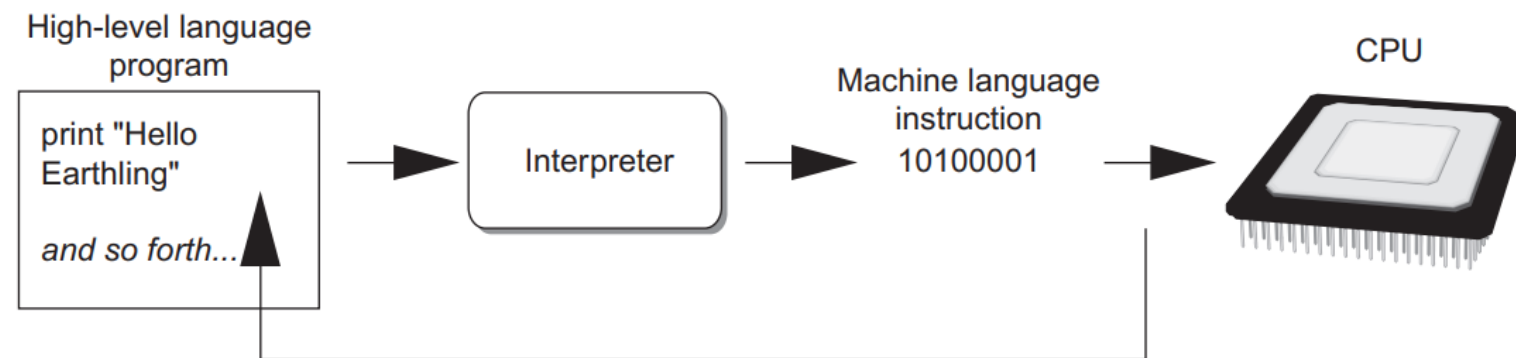




Compilers and Interpreters

• The Python language uses an interpreter

- Which is a program that both translates and executes the instructions in a high-level language program
- The interpreter repeats the following process for every instruction in the program
 - It reads each individual instruction in the program
 - It converts it to machine language instructions
 - It immediately executes them
- Because interpreters combine translation and execution, they typically do not create separate machine language programs



The interpreter translates each high-level instruction to its equivalent machine language instructions and immediately executes them.

This process is repeated for each high-level instruction.



What is Python?

[Hunt 2019]

41

- Python is a general-purpose programming language
 - Originally conceived back in the 1980s by Guido van Rossum at Centrum Wiskunde & Informatica (CWI) in the Netherlands
- Python is now managed by the not-for-profit Python Software Foundation
 - Which was launched in March 2001
 - https://en.wikipedia.org/wiki/Python_Software_Foundation
 - It is also responsible for various processes within the Python community
 - Developing the core Python distribution
 - Managing intellectual rights
 - Supporting developer conferences including PyCon





Why Python?

- Python is popular

- So, lots of examples are available
 - Which makes learning it fast
- The popularity also means that there are lots of modules available for many applications
- Stack Overflow is a Q&A website for professional and enthusiast programmers
 - In February 2020, nearly 65,000 developers participated in their survey to express
 - How they learn and level up
 - Which tools they are using
 - What they want



Why Python?

• Most loved languages

- Python is in the third place



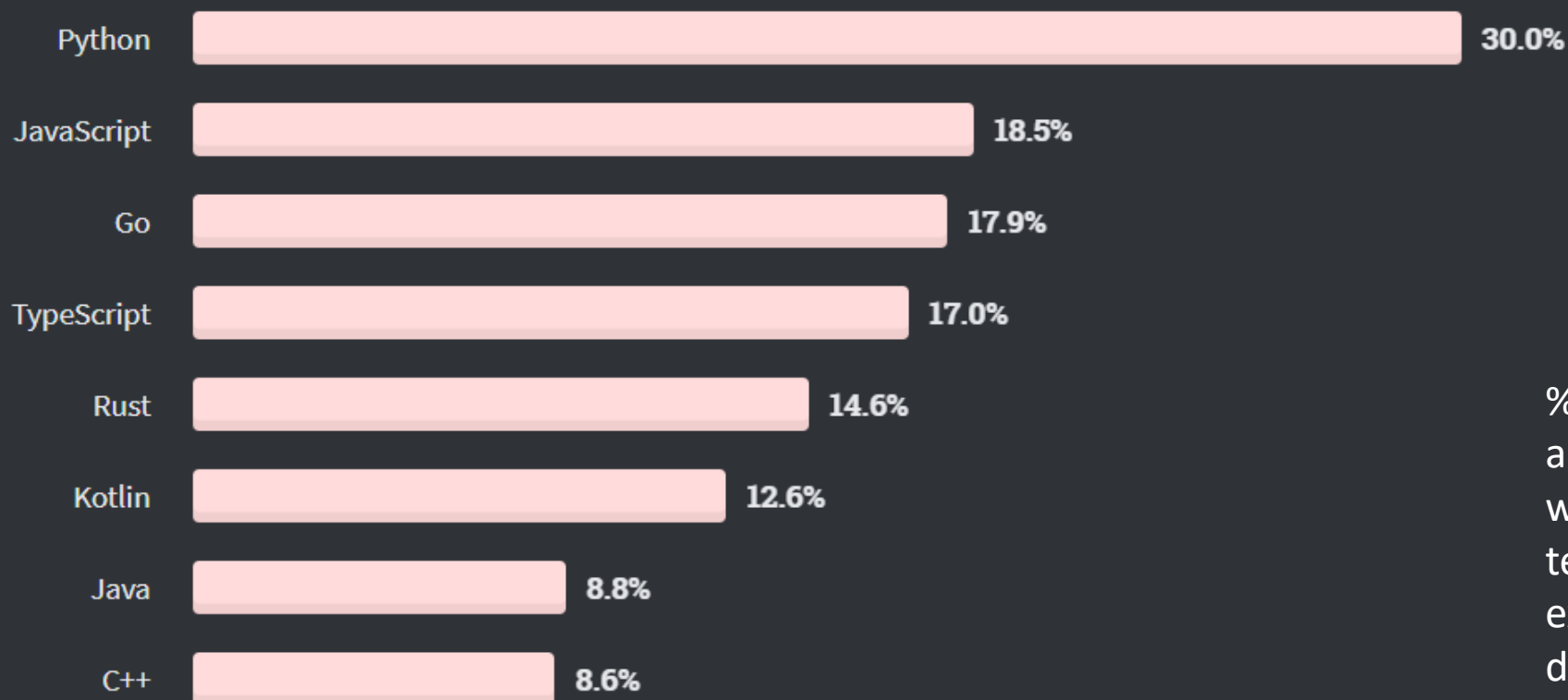
% of developers who are developing with the language or technology and have expressed interest in continuing to develop with it.



Why Python?

- Most wanted languages

- Python takes the top spot for the fourth year in a row



% of developers who are not developing with the language or technology but have expressed interest in developing with it



Why Python?

• Python is simple

- Which again makes it easy to learn
- The clear syntax of Python has earned it the name executable pseudo-code
- The default install of Python already carries high-level data types like lists, tuples, dictionaries, sets, queues, and so on
 - These high-level data types make abstract concepts easy to implement
- The problem with some other languages is that it takes a lot of code to get simple things done

```
#include <stdio.h>
int main(void)
{
    printf("Hello, world!");
}
```

C

```
#include <iostream.h>
int main()
{
    std::cout << "Hello, world! ";
    return 0;
}
```

C++

```
class HelloWorld {
    public static void main(String[]
args) {
        System.out.println("Hello,
World!");
    }
}
```

Java

```
print "Hello, world!"
```

Python



Why Python?

- Python is powerful

- It combines the power of general-purpose programming languages with the ease of use of domain-specific scripting languages like MATLAB or R
- As a general-purpose programming language, Python also allows for
 - The creation of complex graphical user interfaces (GUIs)
 - The creation of web services
 - The integration into existing systems

- Python is interactive

- One of the main advantages of using Python is the ability to interact directly with the code
 - Using a terminal or other tools like the Jupyter Notebook
- Machine learning and data analysis are fundamentally iterative processes, in which the data drives the analysis
 - It is essential for these processes to have tools that allow quick iteration and easy interaction



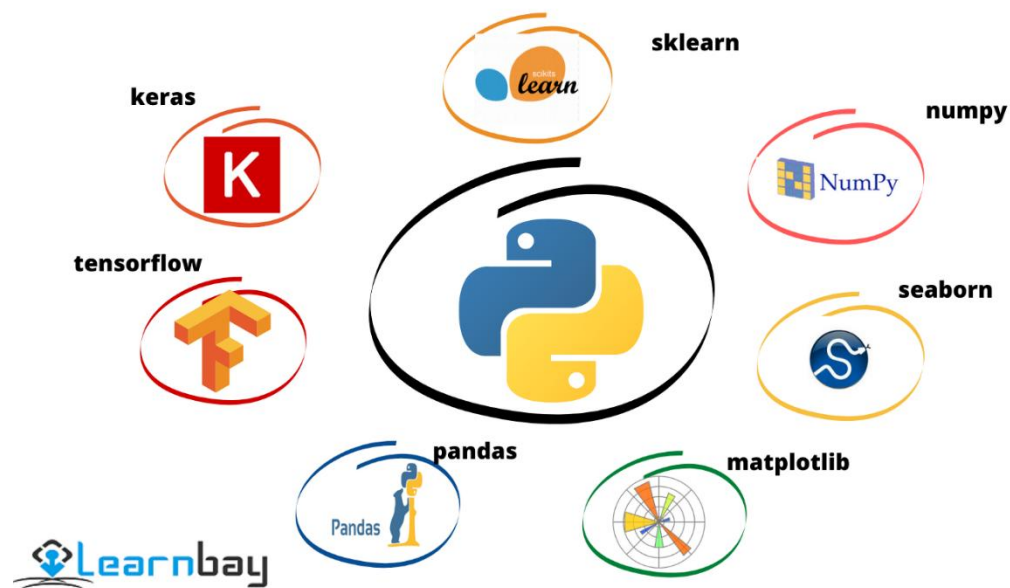
Why Python?

• Python is extensible

- The availability of a wide range of libraries (modules) that can be used to extend the basic features of the language
- The usefulness of Python for data science stems primarily from the large and active ecosystem of third-party packages
- Python has libraries for data loading, visualization, statistics, natural language processing, image processing, and more
 - This vast toolbox provides data scientists with a large array of general- and special-purpose functionality



**One Ring Language
to Rule Them All!**





Why Python?

- Python is free!

- There are high-level languages that allow you to do matrix math such as MATLAB and Mathematica
 - MATLAB has a number of built-in features that make machine learning easier
 - MATLAB is also very fast
- The problem with MATLAB is that to legally use it will cost you a few thousand dollars
 - There are third-party add-ons to MATLAB
 - But nothing on the scale of an open-source project

- Python is cross platform

- It is able to run on (almost) any operating system
 - Particularly the big three operating systems Windows, MacOS, and Linux

MATLAB

- ☒ **EUR 2.000** ⓘ
Perpetual license
- ☐ **EUR 800** ⓘ
Annual license

[Buy now](#)

[—View another product—](#) ▼

Price applies for purchase and use in Germany. For pricing in other regions [contact sales](#). Pricing excludes TAX/VAT.



PyPI and pip

- The Python Package Index (PyPI)

- Is a repository of software for the Python programming language

- pip

- Is a package manager that uses PyPI as the default source for installing packages and their dependencies





Running the First Program on Google Colab

- `print()`

- is a predefined function that prints things out to the output stream
- This output stream of data can be sent to an output window such as the terminal on a Mac or Command Window on a Windows PC
 - In this case, we are printing the string 'Hello World'

- Google Colab

- Colaboratory, or "Colab" for short, allows you to write and execute Python in your browser, with
 - Zero configuration required
 - Free access to GPUs
 - Easy sharing
- <https://colab.research.google.com/>

```
print("Hello world!")
```